NAME: RAMYA S S

REG NO: 192421226

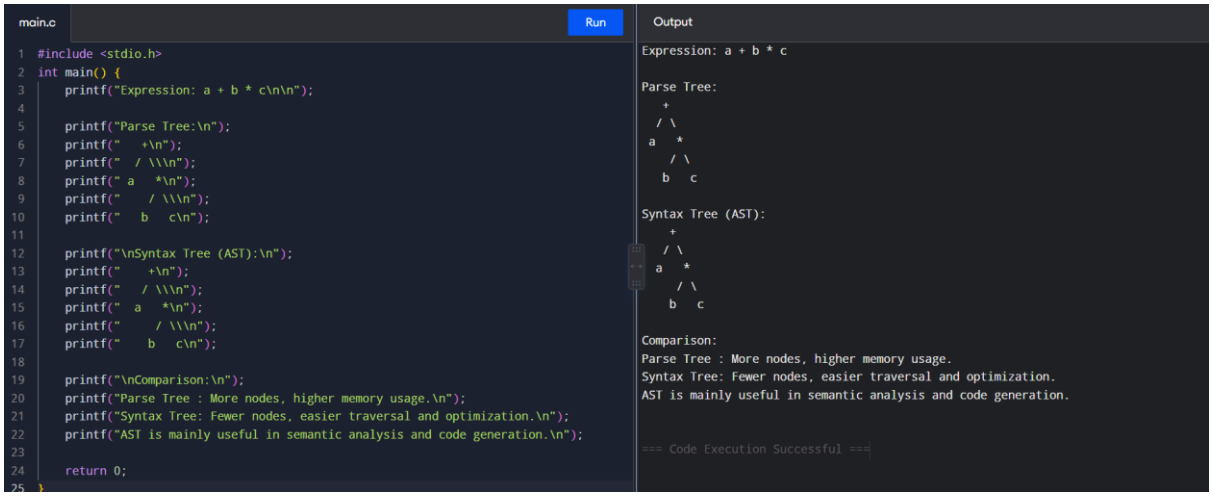
SLOT - D

COURSE: CSA1407 COMPILER DESIGN

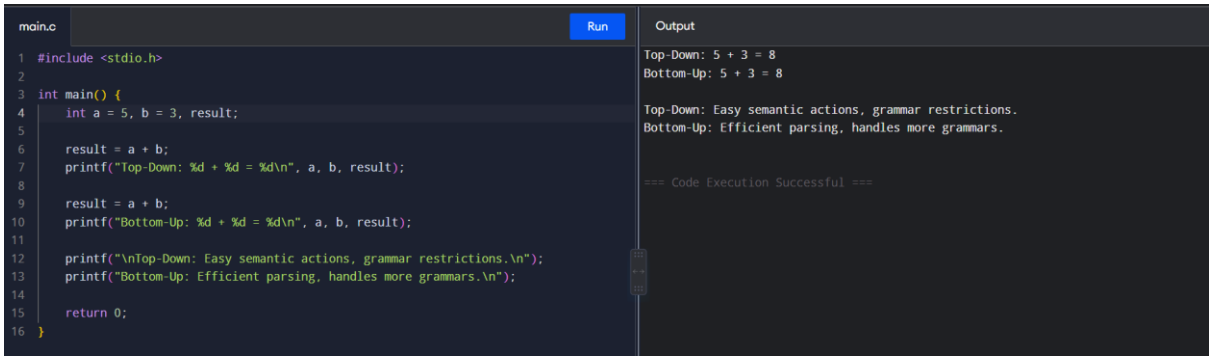
CASE STUDY

SET 1

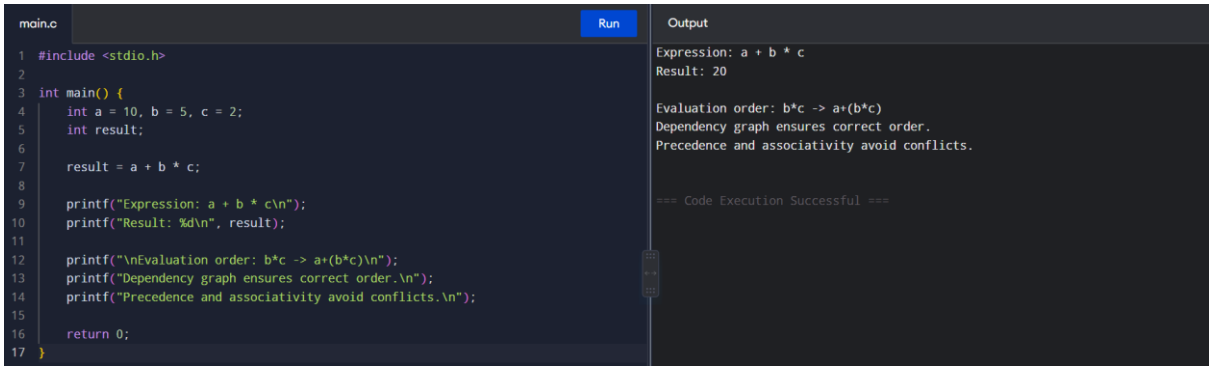
Q1. Parse Tree / Syntax Tree Representation



Q2. Bottom-Up Evaluation



Q3. Attribute Dependency



main.c

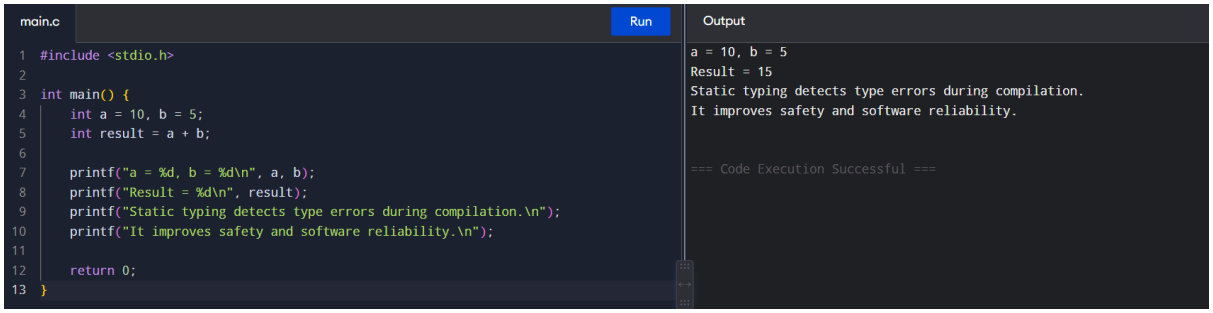
Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5, c = 2;
    int result;

    result = a + b * c;

    printf("Expression: a + b * c\n");
    printf("Result: %d\n", result);

    printf("\nEvaluation order: b*c -> a+(b*c)\n");
    printf("Dependency graph ensures correct order.\n");
    printf("Precedence and associativity avoid conflicts.\n");

    return 0;
}
```

Output

```
Expression: a + b * c
Result: 20

Evaluation order: b*c -> a+(b*c)
Dependency graph ensures correct order.
Precedence and associativity avoid conflicts.


=== Code Execution Successful ===
```

Q4. Static Type Checking



main.c

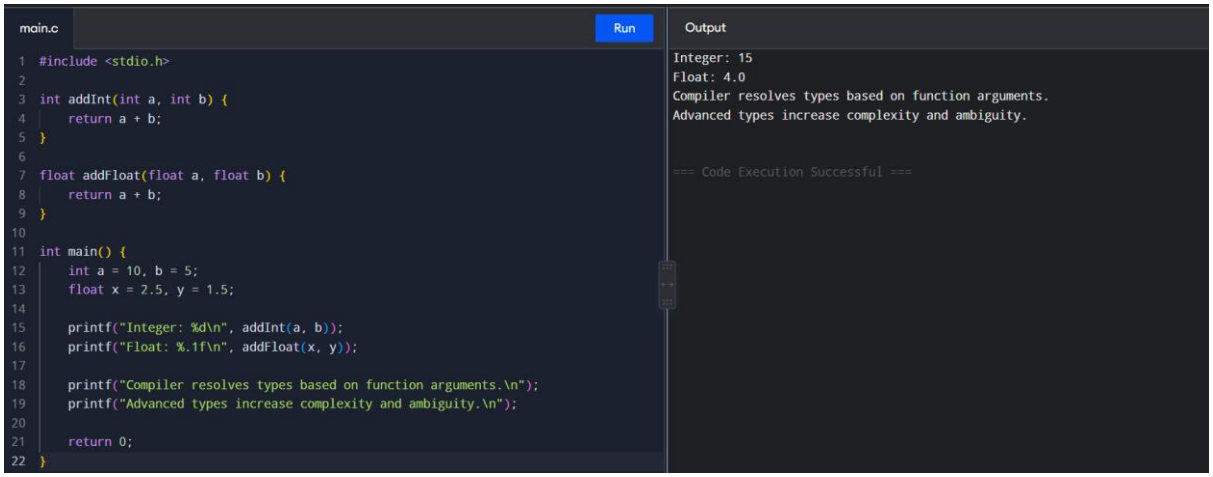
Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    int result = a + b;

    printf("a = %d, b = %d\n", a, b);
    printf("Result = %d\n", result);
    printf("Static typing detects type errors during compilation.\n");
    printf("It improves safety and software reliability.\n");

    return 0;
}
```

Output

```
a = 10, b = 5
Result = 15
Static typing detects type errors during compilation.
It improves safety and software reliability.


=== Code Execution Successful ===
```

Q5. Function Overloading / Type Resolution Demonstration



main.c

Run

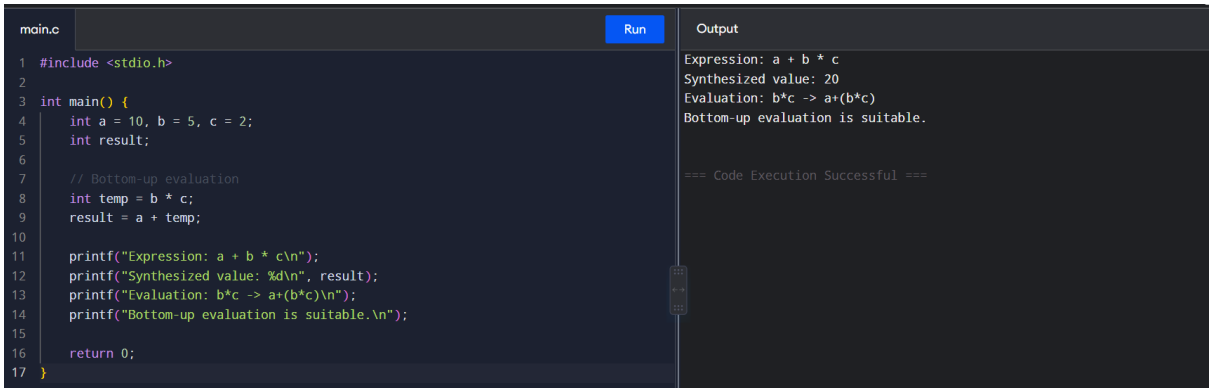
```c
#include <stdio.h>

int addInt(int a, int b) {
    return a + b;
}

float addFloat(float a, float b) {
    return a + b;
}

int main() {
    int a = 10, b = 5;
    float x = 2.5, y = 1.5;

    printf("Integer: %d\n", addInt(a, b));
    printf("Float: %.1f\n", addFloat(x, y));

    printf("Compiler resolves types based on function arguments.\n");
    printf("Advanced types increase complexity and ambiguity.\n");

    return 0;
}
```

Output

```
Integer: 15
Float: 4.0
Compiler resolves types based on function arguments.
Advanced types increase complexity and ambiguity.


=== Code Execution Successful ===
```

SET 2

Q1. Synthesized Attribute



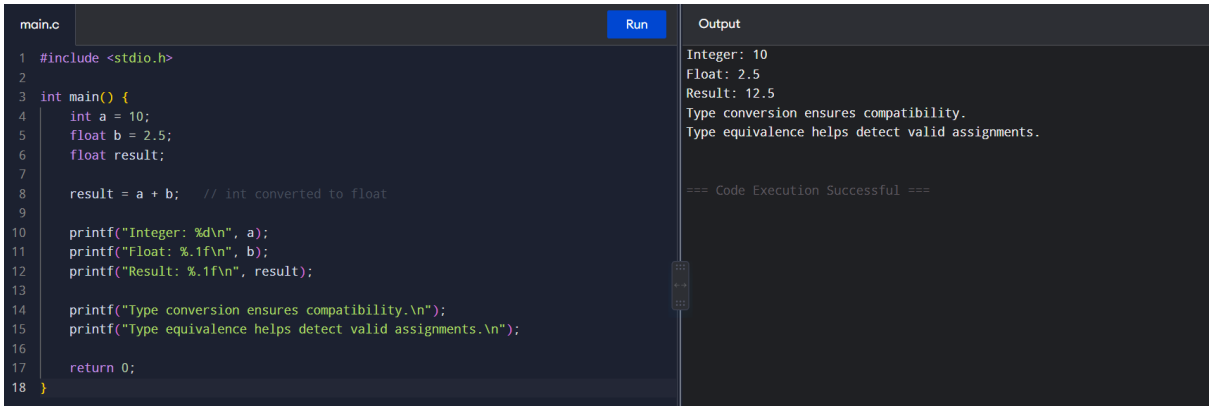
main.c

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5, c = 2;
    int result;

    // Bottom-up evaluation
    int temp = b * c;
    result = a + temp;

    printf("Expression: a + b * c\n");
    printf("Synthesized value: %d\n", result);
    printf("Evaluation: b*c -> a+(b*c)\n");
    printf("Bottom-up evaluation is suitable.\n");

    return 0;
}
```

Output

```
Expression: a + b * c
Synthesized value: 20
Evaluation: b*c -> a+(b*c)
Bottom-up evaluation is suitable.


=== Code Execution Successful ===
```

Q2. Type Conversion



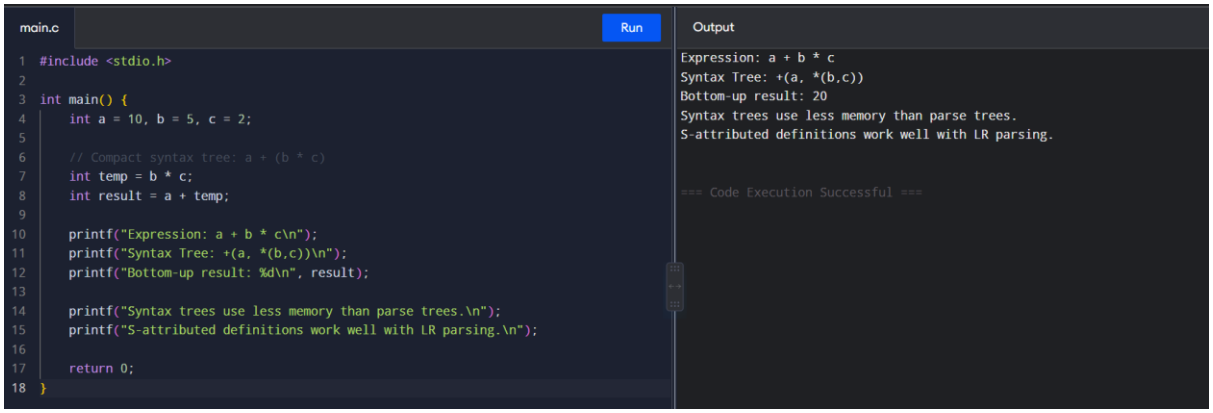
main.c

```c
#include <stdio.h>

int main() {
    int a = 10;
    float b = 2.5;
    float result;

    result = a + b;   // int converted to float

    printf("Integer: %d\n", a);
    printf("Float: %.1f\n", b);
    printf("Result: %.1f\n", result);

    printf("Type conversion ensures compatibility.\n");
    printf("Type equivalence helps detect valid assignments.\n");

    return 0;
}
```

Output

```
Integer: 10
Float: 2.5
Result: 12.5
Type conversion ensures compatibility.
Type equivalence helps detect valid assignments.


=== Code Execution Successful ===
```

Q3. Syntax Tree and Bottom-Up Evaluation



main.c

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5, c = 2;

    // Compact syntax tree: a + (b * c)
    int temp = b * c;
    int result = a + temp;

    printf("Expression: a + b * c\n");
    printf("Syntax Tree: +(a, *(b,c))\n");
    printf("Bottom-up result: %d\n", result);

    printf("Syntax trees use less memory than parse trees.\n");
    printf("S-attributed definitions work well with LR parsing.\n");

    return 0;
}
```

Output

```
Expression: a + b * c
Syntax Tree: +(a, *(b,c))
Bottom-up result: 20
Syntax trees use less memory than parse trees.
S-attributed definitions work well with LR parsing.


=== Code Execution Successful ===
```

Q4. Left-to-Right Attribute Evaluation

main.c

Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    int temp, result;

    // Left-to-right evaluation
    temp = a + b;
    result = temp * 2;

    printf("Expression: (a + b) * 2\n");
    printf("Intermediate code:\n");
    printf("t1 = a + b\n");
    printf("t2 = t1 * 2\n");
    printf("Result = %d\n", result);

    printf("L-attributed evaluation: left to right.\n");
    printf("Dependency management ensures correct order.\n");

    return 0;
}
```

Output

```
Expression: (a + b) * 2
Intermediate code:
t1 = a + b
t2 = t1 * 2
Result = 30
L-attributed evaluation: left to right.
Dependency management ensures correct order.


=== Code Execution Successful ===
```

Q5. Type Checking

main.c

Run

```c
#include <stdio.h>

int main() {
    int a = 10;
    float b = 5.5;

    float x = a;       // int to float
    int y = (int)b;    // float to int

    printf("Integer: %d\n", a);
    printf("Float: %.1f\n", b);
    printf("Int to Float: %.1f\n", x);
    printf("Float to Int: %d\n", y);

    printf("Type checking ensures compatibility.\n");
    printf("Type equivalence improves reliable compilation.\n");

    return 0;
}
```

Output

```
Integer: 10
Float: 5.5
Int to Float: 10.0
Float to Int: 5
Type checking ensures compatibility.
Type equivalence improves reliable compilation.


=== Code Execution Successful ===
```

SET 3

Q1. Syntax-Directed Definition – Expression Evaluation

main.c

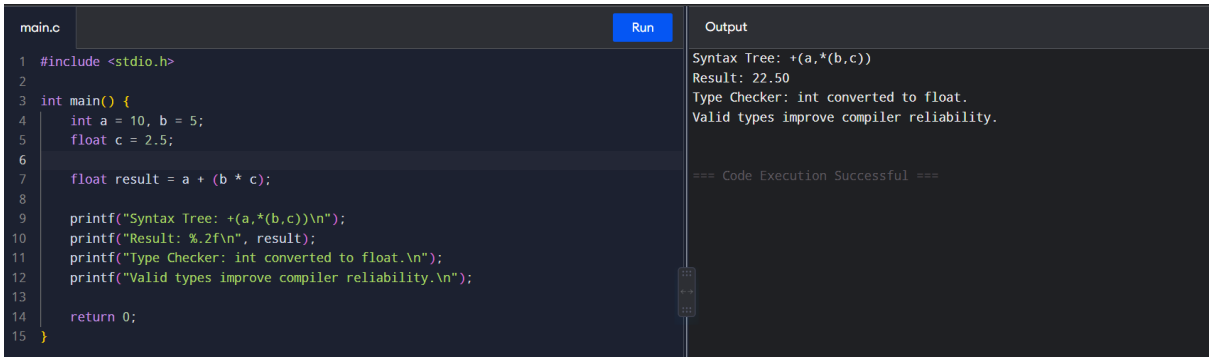
Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // S-attributed: bottom-up evaluation
    int sum = a + b;

    // L-attributed: left-to-right evaluation
    float result = sum * c;

    printf("Syntax Tree: *(+(a,b),c)\n");
    printf("S-attributed result: %d\n", sum);
    printf("L-attributed result: %.2f\n", result);
    printf("Type conversion: int -> float\n");

    return 0;
}
```

Output

```
Syntax Tree: *(+(a,b),c)
S-attributed result: 15
L-attributed result: 37.50
Type conversion: int -> float


=== Code Execution Successful ===
```

Q2. Type Checker with Type Conversion



main.c

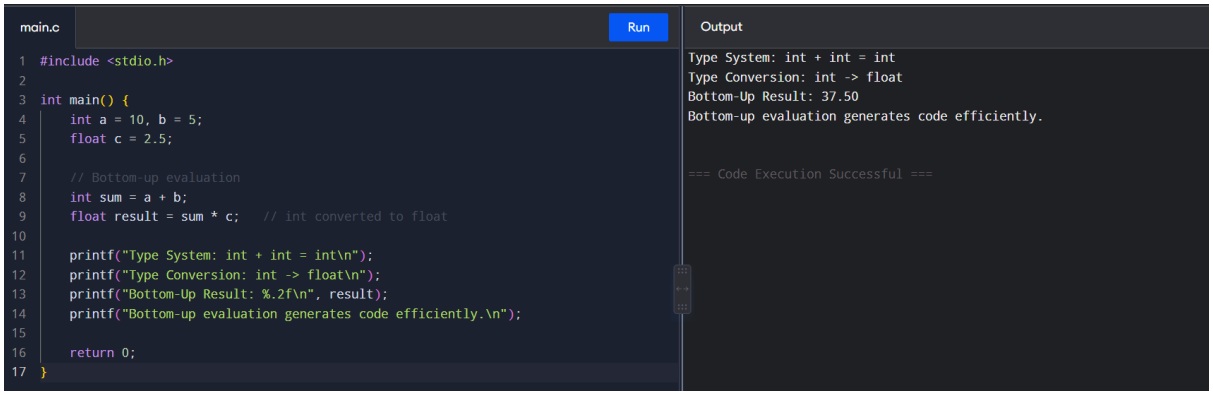
Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    float result = a + (b * c);

    printf("Syntax Tree: +(a,*(b,c))\n");
    printf("Result: %.2f\n", result);
    printf("Type Checker: int converted to float.\n");
    printf("Valid types improve compiler reliability.\n");

    return 0;
}
```

Output

```
Syntax Tree: +(a,*(b,c))
Result: 22.50
Type Checker: int converted to float.
Valid types improve compiler reliability.


=== Code Execution Successful ===
```

Q3. Bottom-Up Expression Evaluation



main.c

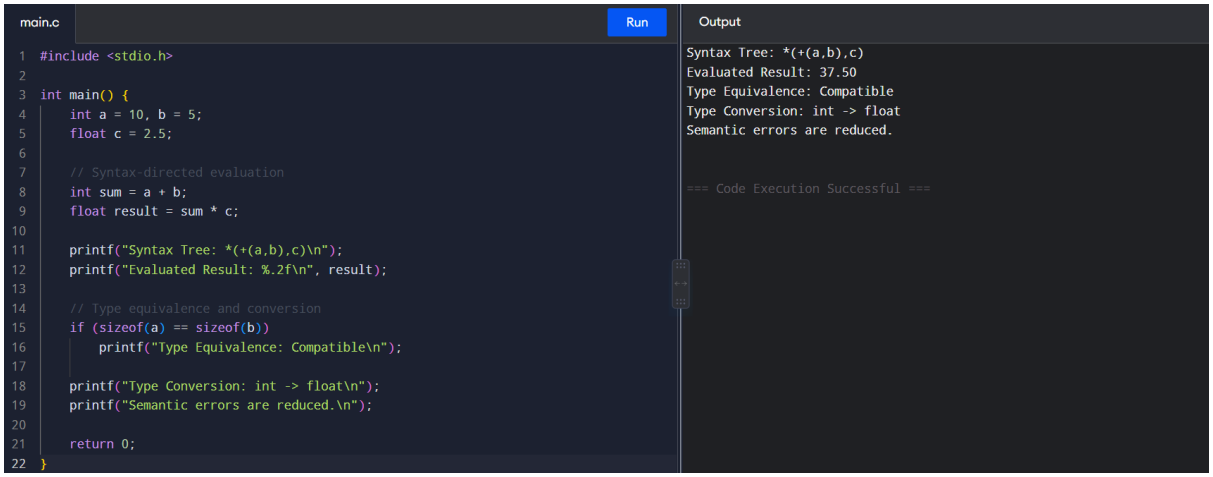
Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // Bottom-up evaluation
    int sum = a + b;
    float result = sum * c;    // int converted to float

    printf("Type System: int + int = int\n");
    printf("Type Conversion: int -> float\n");
    printf("Bottom-Up Result: %.2f\n", result);
    printf("Bottom-up evaluation generates code efficiently.\n");

    return 0;
}
```

Output

```
Type System: int + int = int
Type Conversion: int -> float
Bottom-Up Result: 37.50
Bottom-up evaluation generates code efficiently.


=== Code Execution Successful ===
```

Q4. Type Equivalence Checking



main.c

Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // Syntax-directed evaluation
    int sum = a + b;
    float result = sum * c;

    printf("Syntax Tree: *(+(a,b),c)\n");
    printf("Evaluated Result: %.2f\n", result);

    // Type equivalence and conversion
    if (sizeof(a) == sizeof(b))
        printf("Type Equivalence: Compatible\n");

    printf("Type Conversion: int -> float\n");
    printf("Semantic errors are reduced.\n");

    return 0;
}
```

Output

```
Syntax Tree: *(+(a,b),c)
Evaluated Result: 37.50
Type Equivalence: Compatible
Type Conversion: int -> float
Semantic errors are reduced.


=== Code Execution Successful ===
```

Q5. Type Checking and Type Conversion

main.c

Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // Bottom-up evaluation
    int sum = a + b;
    float result = sum * c;

    printf("Top-Down: Simple but complex for nested dependencies.\n");
    printf("Bottom-Up: Efficient for complex expressions.\n");
    printf("Syntax Tree: *(+(a,b),c)\n");

    printf("Type Checking: int and float are compatible.\n");
    printf("Type Conversion: int -> float\n");
    printf("Result: %.2f\n", result);

    return 0;
}
```

Output

```
Top-Down: Simple but complex for nested dependencies.
Bottom-Up: Efficient for complex expressions.
Syntax Tree: *(+(a,b),c)
Type Checking: int and float are compatible.
Type Conversion: int -> float
Result: 37.50


=== Code Execution Successful ===
```

SET 4

Q1. Syntax-Directed Definition and Type Conversion

main.c

Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // Syntax-directed evaluation
    int sum = a + b;
    float result = sum * c;

    printf("Syntax Tree: *(+(a,b),c)\n");
    printf("Result: %.2f\n", result);

    // Type conversion
    float x = a;
    printf("Type Conversion: int -> float = %.1f\n", x);

    printf("Syntax-directed definitions build syntax trees.\n");
    printf("Type conversion ensures compatible operations.\n");

    return 0;
}
```

Output

```
Syntax Tree: *(+(a,b),c)
Result: 37.50
Type Conversion: int -> float = 10.0
Syntax-directed definitions build syntax trees.
Type conversion ensures compatible operations.


=== Code Execution Successful ===
```

Q2. Top-Down and Bottom-Up Evaluation

main.c

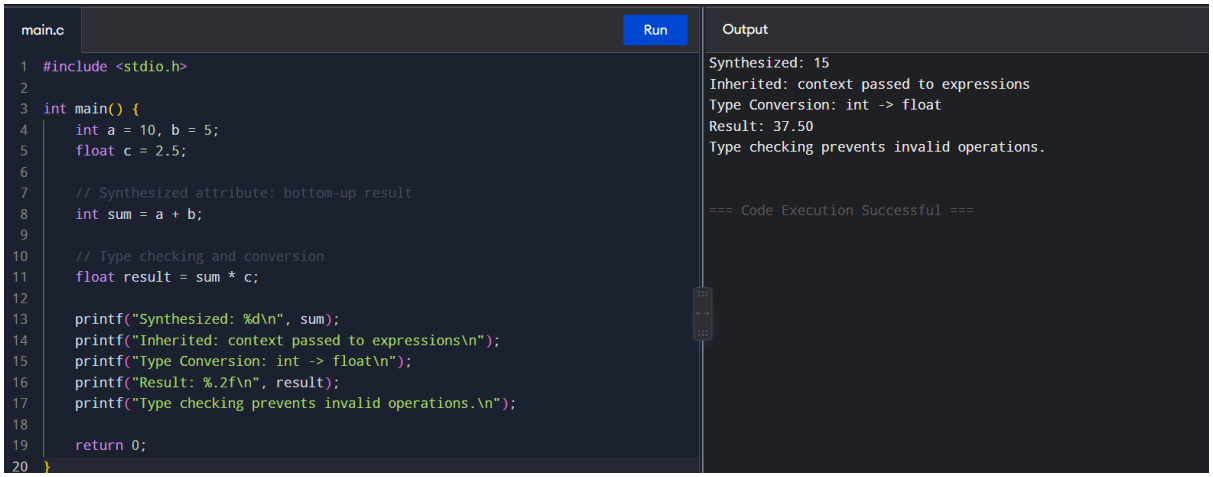
Run

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float x = 2.5;

    // Bottom-up evaluation
    int sum = a + b;
    float result = sum * x;

    printf("Top-Down: Starts from root and moves downward.\n");
    printf("Bottom-Up: Evaluates operands and moves upward.\n");
    printf("Result: %.2f\n", result);

    // Type equivalence
    if (sizeof(a) == sizeof(b))
        printf("Type Equivalence: Compatible\n");

    printf("Type checking prevents unsafe assignments.\n");

    return 0;
}
```

Output

```
Top-Down: Starts from root and moves downward.
Bottom-Up: Evaluates operands and moves upward.
Result: 37.50
Type Equivalence: Compatible
Type checking prevents unsafe assignments.


=== Code Execution Successful ===
```

Q3. Inherited and Synthesized Attributes



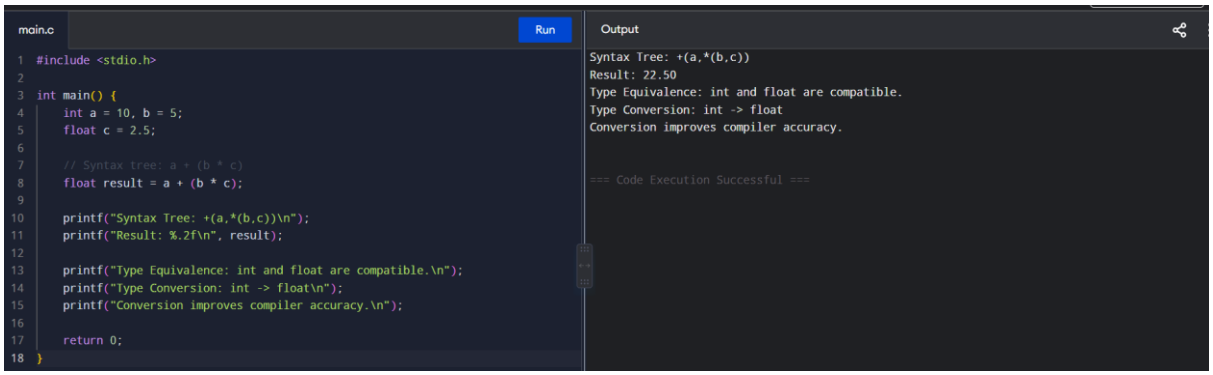
main.c

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // Synthesized attribute: bottom-up result
    int sum = a + b;

    // Type checking and conversion
    float result = sum * c;

    printf("Synthesized: %d\n", sum);
    printf("Inherited: context passed to expressions\n");
    printf("Type Conversion: int -> float\n");
    printf("Result: %.2f\n", result);
    printf("Type checking prevents invalid operations.\n");

    return 0;
}
```

Output

```
Synthesized: 15
Inherited: context passed to expressions
Type Conversion: int -> float
Result: 37.50
Type checking prevents invalid operations.


=== Code Execution Successful ===
```

Q4. Type Equivalence and Type Conversion



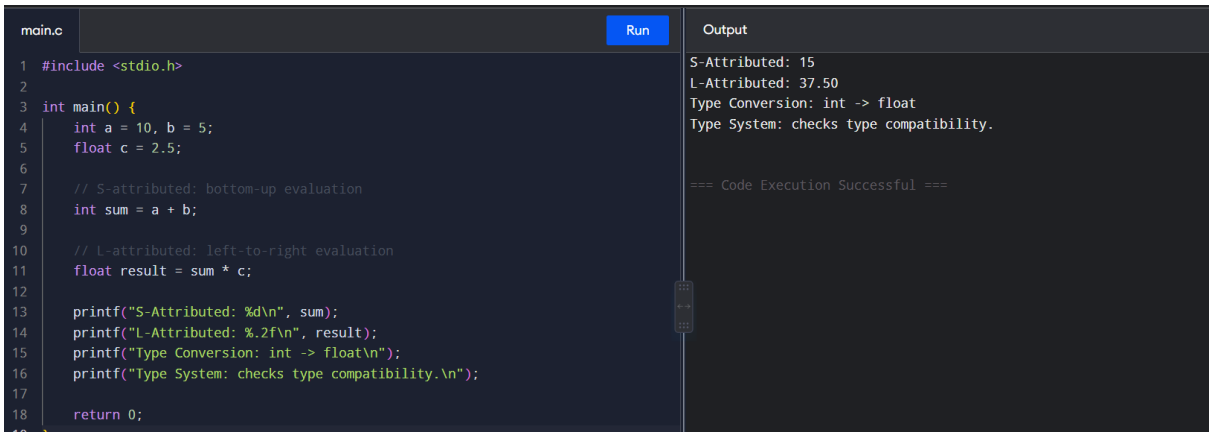
main.c

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // Syntax tree: a + (b * c)
    float result = a + (b * c);

    printf("Syntax Tree: +(a,*(b,c))\n");
    printf("Result: %.2f\n", result);

    printf("Type Equivalence: int and float are compatible.\n");
    printf("Type Conversion: int -> float\n");
    printf("Conversion improves compiler accuracy.\n");

    return 0;
}
```

Output

```
Syntax Tree: +(a,*(b,c))
Result: 22.50
Type Equivalence: int and float are compatible.
Type Conversion: int -> float
Conversion improves compiler accuracy.


=== Code Execution Successful ===
```

Q5. S-Attributed and L-Attributed Definitions



main.c

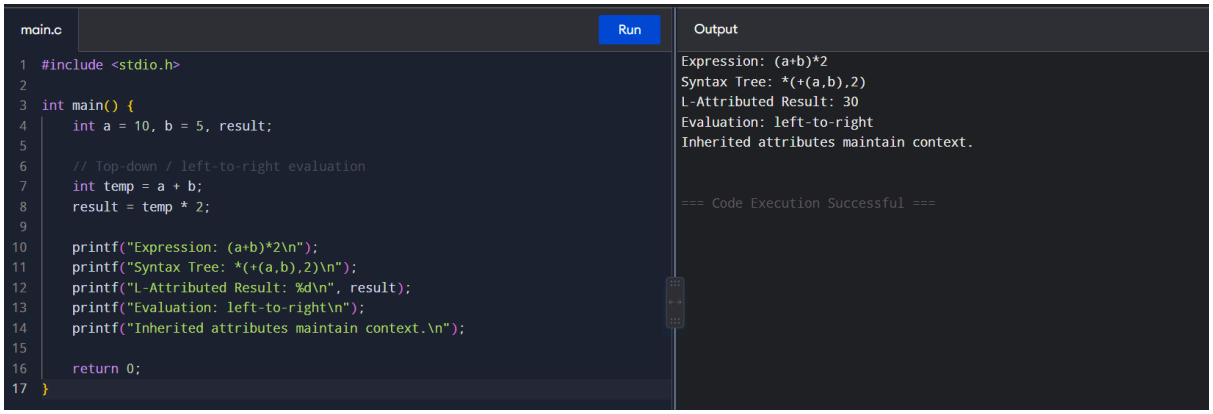
```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    float c = 2.5;

    // S-attributed: bottom-up evaluation
    int sum = a + b;

    // L-attributed: left-to-right evaluation
    float result = sum * c;

    printf("S-Attributed: %d\n", sum);
    printf("L-Attributed: %.2f\n", result);
    printf("Type Conversion: int -> float\n");
    printf("Type System: checks type compatibility.\n");

    return 0;
```

Output

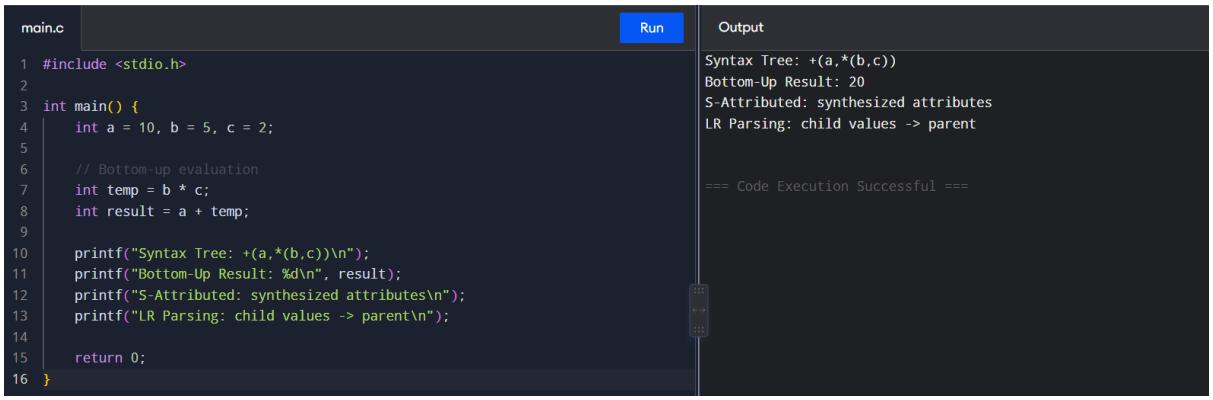
```
S-Attributed: 15
L-Attributed: 37.50
Type Conversion: int -> float
Type System: checks type compatibility.


=== Code Execution Successful ===
```

SET 5

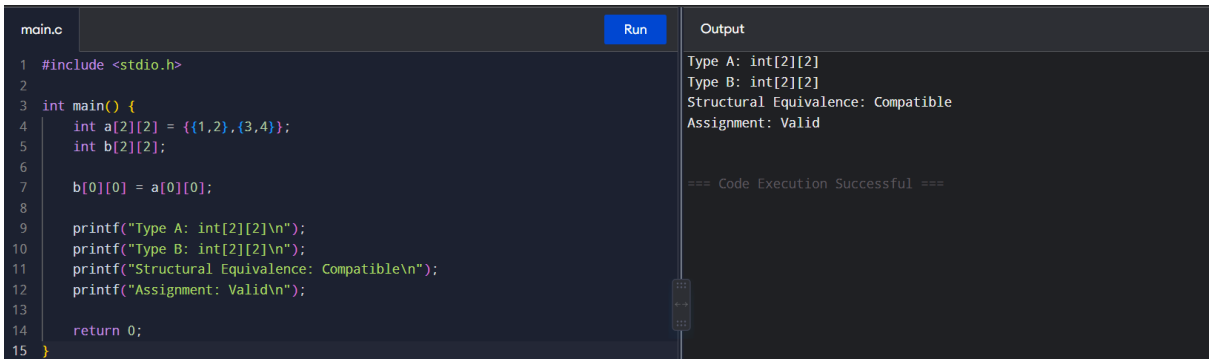
Q1. L-Attributed Definition



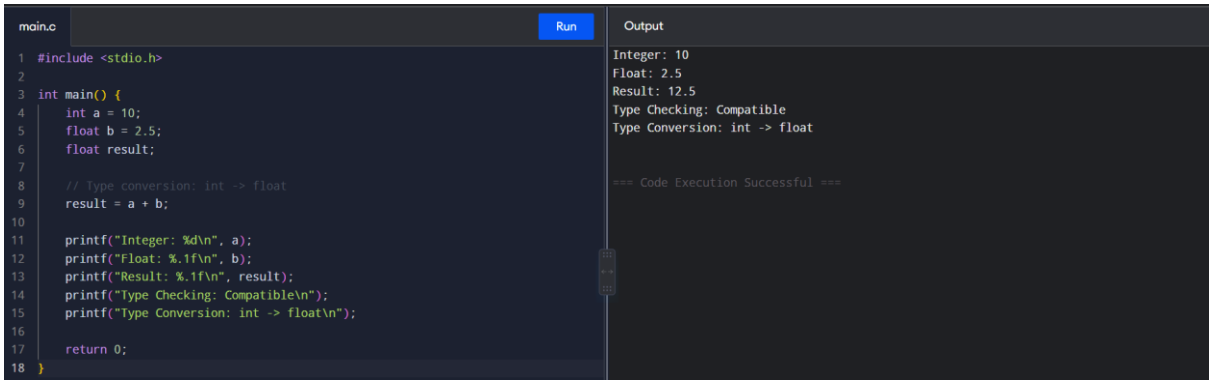
Q2. Bottom-Up Evaluation



Q3. Type Equivalence



Q4. Type Checking and Type Conversion



Q5. Attribute Dependency and Expression Evaluation